

ORIE 5135 — Integer Programming Project

Exam Scheduling via ILP

Zijian Zou (NetID: zz929)

Instructor: Professor Andrea Lodi

May 8, 2026

Contents

1	Question 1: Natural and Clique-Strengthened ILP	2
1.1	Question 1a: Natural Formulation (F1)	2
1.2	Question 1b: Clique-Strengthened Formulation (F2)	2
2	Question 2: Extended Formulation (F3)	3
2.1	Question 2a: Set-Partitioning Formulation	3
2.2	Question 2b: Set Covering vs. Set Partitioning	3
3	Question 3: Computational Results	3
3.1	Question 3a: Dataset 1 (120 s limit)	4
3.2	Question 3b: Dataset 2 (600 s limit)	5

1 Question 1: Natural and Clique-Strengthened ILP

1.1 Question 1a: Natural Formulation (F1)

Decision variables

$$\begin{aligned} x_{ij} &\in \{0, 1\}, \quad i = 1, \dots, m, \quad j = 1, \dots, n && (1 \text{ if exam } i \text{ is scheduled in slot } j) \\ y_j &\in \{0, 1\}, \quad j = 1, \dots, n && (1 \text{ if slot } j \text{ is used}) \end{aligned}$$

Formulation

$$\min \sum_{j=1}^n y_j \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^n x_{ij} = 1 \quad \forall i = 1, \dots, m \tag{2}$$

$$\sum_{i=1}^m r_i x_{ij} \leq R y_j \quad \forall j = 1, \dots, n \tag{3}$$

$$x_{ij} + x_{i'j} \leq 1 \quad \forall (i, i') \in E, \quad j = 1, \dots, n \tag{4}$$

$$y_j \geq y_{j+1} \quad \forall j = 1, \dots, n-1 \tag{5}$$

$$x_{ij}, y_j \in \{0, 1\} \quad \forall i, j$$

Constraint explanation

- (2): every exam is assigned to exactly one slot.
- (3): enforces the proctor budget per slot and links x to y .
- (4): two conflicting exams cannot share a slot.
- (5): symmetry-breaking — slots are filled from index 1 upward, eliminating permutation symmetry among unused slots.

1.2 Question 1b: Clique-Strengthened Formulation (F2)

Let $C \subseteq V$ be a clique in G . Since every pair in C conflicts, at most one exam in C can occupy any slot, giving the *clique constraint*:

$$\sum_{i \in C} x_{ij} \leq 1 \quad \forall j = 1, \dots, n.$$

For a clique of size k , this single inequality dominates all $\binom{k}{2}$ pairwise constraints (4). It is strictly tighter in the LP relaxation for $k \geq 3$: setting $x_{ij} = \frac{1}{k-1}$ satisfies every pairwise constraint yet violates the clique constraint (since $\frac{k}{k-1} > 1$). It suffices to use only maximal cliques, since every edge belongs to at least one.

Formulation (F2): replace (4) with

$$\sum_{i \in C} x_{ij} \leq 1 \quad \forall C \in \mathcal{C}_{\max}(G), \quad j = 1, \dots, n. \tag{6}$$

All other constraints and variables are identical to F1.

2 Question 2: Extended Formulation (F3)

2.1 Question 2a: Set-Partitioning Formulation

A subset $S \subseteq \{1, \dots, m\}$ is a *feasible grouping* if $\sum_{i \in S} r_i \leq R$ and S is a stable set in G . Let $\mathcal{A}$ be the collection of all non-empty feasible groupings, and let $\lambda_S \in \{0, 1\}$ indicate whether grouping S is selected as a time slot.

$$\min \sum_{S \in \mathcal{A}} \lambda_S \tag{7}$$

$$\begin{aligned} \text{s.t. } \sum_{\substack{S \in \mathcal{A} \\ i \in S}} \lambda_S &= 1 \forall i = 1, \dots, m \\ \lambda_S &\in \{0, 1\} \forall S \in \mathcal{A} \end{aligned} \tag{8}$$

The objective counts selected groupings and constraint (8) ensures each exam appears in exactly one. The proctor budget and conflict requirements are already handled by how $\mathcal{A}$ was defined — any $S \in \mathcal{A}$ is feasible by construction, so no extra constraints are needed.

2.2 Question 2b: Set Covering vs. Set Partitioning

For constraint (8) there are two natural options:

Set partitioning ($= 1$): each exam appears in exactly one selected grouping, which is what the problem actually requires. Because it is an equality, the LP relaxation cannot spread fractional λ values across multiple groupings, so the bound tends to be tighter.

Set covering (≥ 1): each exam needs to be covered at least once. The LP can exploit this slack by using fractional values summing to slightly above 1, reporting a smaller objective than set partitioning would allow. The LP bound is therefore no better — and often weaker.

For this problem set partitioning is the right choice. It matches the problem exactly and gives a tighter LP bound, which means less branching. Set covering does have one advantage in column-generation settings: adding a new column can never make a covering constraint infeasible, which makes the column-generation subproblem a bit easier to manage. Since we enumerate all groupings upfront for Dataset 1 that advantage does not apply here.

3 Question 3: Computational Results

All experiments use `gurobipy` (Gurobi 12.0.3) on an Apple M2 (8 cores). **Opt** = solved to optimality within the time limit; **TL** = time limit reached (objective shown is the best feasible solution found).

3.1 Question 3a: Dataset 1 (120 s limit)

Table 1: *

Table 1. Dataset 1 — LP relaxation values and optimality status.

Instance	m	LP (F1)	LP (F2)	LP (F3)	Obj	F1 / F2 / F3
instance_01	35	6.32	6.32	7.18	8	Opt / Opt / Opt
instance_02	38	7.32	7.32	7.92	9	Opt / Opt / Opt
instance_03	40	7.88	7.88	8.12	9	Opt / Opt / Opt
instance_04	60	12.84	12.84	12.84	13	TL / Opt / Opt
instance_05	63	12.40	12.40	12.40	13	Opt / Opt / Opt
instance_06	65	12.88	12.88	12.88	13	TL / TL / Opt
instance_07	68	12.68	12.68	12.68	13	Opt / Opt / Opt
instance_08	75	15.36	15.36	15.36	16	Opt / Opt / Opt
instance_09	75	14.16	14.16	14.16	15	Opt / Opt / Opt
instance_10	75	14.68	14.68	14.68	15	Opt / TL / Opt

Table 2: *

Table 2. Dataset 1 — branch-and-bound node counts and solve times.

Instance	m	F1		F2		F3	
		Nodes	Time (s)	Nodes	Time (s)	Nodes	Time (s)
instance_01	35	1	2.73	1	5.79	1	0.03
instance_02	38	904	2.62	561	7.80	1	0.09
instance_03	40	140	2.01	154	7.24	1	0.14
instance_04	60	79831	120.02	3041	90.79	1	0.76
instance_05	63	1	14.46	1	27.63	1	0.52
instance_06	65	29462	120.08	2826	120.05	413	4.11
instance_07	68	3331	27.19	3009	117.08	1	2.83
instance_08	75	1	20.55	1	40.25	1	0.62
instance_09	75	1	6.49	1	106.31	1	1.01
instance_10	75	1446	29.91	1	120.03	1	0.81

Discussion

(i) LP relaxation comparison. F1 and F2 give the same LP bound on every single instance. The reason is that Gurobi already adds clique cuts automatically at the root node of F1, so by the time the root LP is solved the explicit clique constraints in F2 are redundant — they add nothing new. F3 does better on the small instances: for example, 7.18 vs. 6.32 on instance_01 and 8.12 vs. 7.88 on instance_03. On medium and large instances all three formulations happen to give the

same LP value.

(ii) Equal LP bound does not imply equal B&B effort. Instances 09 and 10 show this well. On instance_09 all three formulations share $LP = 14.16$, yet F1 finishes in 6.5s while F2 takes 106s. Both explore only the root node, but each LP re-solve inside Gurobi’s cut loop is slower for F2 because of the extra $|\mathcal{C}_{\max}| \times n$ rows. Instance_10 is even more telling: F1 finds the optimum of 15 in 30s after 1446 nodes, while F2 runs out of time at just 1 node and returns an incumbent of 16. The additional clique constraints slow down every single LP solve so much that F2 never gets to explore the tree. This shows that how expensive each node is can matter just as much as how tight the bound is.

(iii) Scaling with instance size. F3 handles the larger instances best. It solves 9 out of 10 at the root node in under 5s, with grouping enumeration adding less than a second on top. F1 is reasonable up to $m = 68$ but struggles on instances 04 and 06, where it needs tens of thousands of nodes and still times out on 04. F2 consistently runs slower than F1 and also fails on instances 06 and 10 — as m grows the number of clique constraints grows too, and each LP becomes progressively harder to solve.

3.2 Question 3b: Dataset 2 (600 s limit)

Table 3: *
Table 3. Dataset 2 results (F1 and F2 only).

Instance	m	F1				F2			
		LP	Obj	Nodes	Status	LP	Obj	Nodes	Status
instance_01	80	16.32	17	1	Opt	16.32	17	1	Opt
instance_02	82	16.08	17	1	Opt	16.08	17	1	Opt
instance_03	83	16.28	17	1	Opt	16.28	17	1	Opt
instance_04	87	16.76	17	1	Opt	16.76	18	1	TL
instance_05	87	16.60	17	523	Opt	16.60	18	1	TL
instance_06	87	17.76	18	2295	Opt	17.76	19	1	TL
instance_07	82	16.88	18	118886	TL	16.88	18	2096	TL
instance_08	85	16.84	18	100675	TL	16.84	18	1	TL
instance_09	90	18.00	19	28127	TL	18.00	20	1	TL

Discussion

Computational limits of F1 and F2. The LP bounds for F1 and F2 are again identical throughout. **F1** handles all small ($m = 80$ – 83) and medium ($m = 87$) instances, finishing within 55s, but times out on instances 07–09 after exploring up to 119 000 nodes. **F2** only gets through the small instances; from the medium group onward it spends the entire 600s at the root node without branching at all. At $m \approx 87$ – 90 the number of maximal cliques is large enough that the LP matrix becomes very expensive to solve repeatedly, which is the same issue we saw on Dataset 1

but worse. F2 ends up finding worse solutions than F1 despite having the same LP bound, simply because it cannot afford to search the tree.

Why F3 is not applicable to Dataset 2. F3 needs all feasible groupings enumerated upfront. In the worst case the number of maximal independent sets is exponential in m , and for $m = 80$ – 90 we found in practice that the enumeration just does not finish in time. To apply F3 at this scale one would need to generate groupings on demand instead — essentially a column generation approach, where a weighted maximum stable-set subproblem is solved at each node to find the next useful grouping.